

# 第 18 讲: Guest Lecture: Dan Fu 与推理引擎研究

CS336 Spring 2026 public videos and official course materials

## 目录

|                                                 |          |
|-------------------------------------------------|----------|
| <b>1 本章导读</b>                                   | <b>4</b> |
| 1.1 本章知识路线                                      | 4        |
| 1.2 结构图                                         | 4        |
| <b>2 本章主线</b>                                   | <b>4</b> |
| 2.1 本章要解决的核心问题                                  | 5        |
| 2.2 从机制到资源账本                                    | 5        |
| 2.3 从课堂讲解到可复现实验                                 | 6        |
| <b>3 术语、符号与问题边界</b>                             | <b>6</b> |
| 3.1 关键词                                         | 6        |
| <b>4 从训练到 serving 的另一侧</b>                      | <b>6</b> |
| 4.1 为什么要讨论从训练到 serving 的另一侧                     | 6        |
| 4.2 从训练到 serving 的另一侧的机制                        | 7        |
| 4.3 从训练到 serving 的另一侧的数学表达                      | 7        |
| 4.4 从训练到 serving 的另一侧在系统中的实现                    | 7        |
| 4.5 边界条件与常见误解                                   | 7        |
| 4.6 与本章其他概念的关系                                  | 8        |
| 4.7 怎样检验从训练到 serving 的另一侧                       | 8        |
| 4.8 常见混淆: 从训练到 serving 的另一侧                     | 8        |
| 4.9 从训练到 serving 的另一侧的小结                        | 8        |
| <b>5 GPU kernels 与 full-stack innovation</b>    | <b>8</b> |
| 5.1 为什么要讨论 GPU kernels 与 full-stack innovation  | 9        |
| 5.2 GPU kernels 与 full-stack innovation 的机制     | 9        |
| 5.3 GPU kernels 与 full-stack innovation 的资源账本   | 9        |
| 5.4 GPU kernels 与 full-stack innovation 在系统中的实现 | 9        |
| 5.5 边界条件与常见误解                                   | 9        |

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| 5.6       | 与本章其他概念的关系                                  | 10        |
| 5.7       | 怎样检验 GPU kernels 与 full-stack innovation    | 10        |
| 5.8       | 常见混淆: GPU kernels 与 full-stack innovation   | 10        |
| 5.9       | GPU kernels 与 full-stack innovation 的小结     | 10        |
| <b>6</b>  | <b>SSM/recurrence 与 inference 研究问题</b>      | <b>11</b> |
| 6.1       | 为什么要讨论 SSM/recurrence 与 inference 研究问题      | 11        |
| 6.2       | SSM/recurrence 与 inference 研究问题的机制          | 11        |
| 6.3       | SSM/recurrence 与 inference 研究问题的资源账本        | 11        |
| 6.4       | SSM/recurrence 与 inference 研究问题在系统中的实现      | 11        |
| 6.5       | 边界条件与常见误解                                   | 12        |
| 6.6       | 与本章其他概念的关系                                  | 12        |
| 6.7       | 怎样检验 SSM/recurrence 与 inference 研究问题        | 12        |
| 6.8       | 常见混淆: SSM/recurrence 与 inference 研究问题       | 12        |
| 6.9       | SSM/recurrence 与 inference 研究问题的小结          | 13        |
| <b>7</b>  | <b>综合讨论: 从局部机制到完整系统</b>                     | <b>13</b> |
| 7.1       | 从训练到 serving 的另一侧在系统中的位置                    | 13        |
| 7.2       | GPU kernels 与 full-stack innovation 在系统中的位置 | 13        |
| 7.3       | SSM/recurrence 与 inference 研究问题在系统中的位置      | 13        |
| 7.4       | 把本章知识用于读论文和复现实验                             | 14        |
| <b>8</b>  | <b>例题: 把概念变成可计算检查</b>                       | <b>14</b> |
| 8.1       | Serving latency 的分解                         | 14        |
| <b>9</b>  | <b>实验设计: 从小例子到可复现结论</b>                     | <b>14</b> |
| 9.1       | 最小输入                                        | 15        |
| 9.2       | 资源账本                                        | 15        |
| 9.3       | 单变量消融                                       | 15        |
| 9.4       | 失败条件                                        | 15        |
| 9.5       | 记录与报告                                       | 15        |
| 9.6       | 从小实验到大结论                                    | 15        |
| <b>10</b> | <b>课程资料与回看路径</b>                            | <b>15</b> |
| 10.1      | 视频回看路径                                      | 16        |
| 10.2      | 课程资料阅读路径                                    | 16        |
| <b>11</b> | <b>实践说明、历史位置与风险</b>                         | <b>16</b> |

|                     |           |
|---------------------|-----------|
| <b>12 复习要点</b>      | <b>17</b> |
| 12.1 综合自测 . . . . . | 17        |
| <b>13 总结与延伸</b>     | <b>17</b> |
| 13.1 本章小结 . . . . . | 17        |
| 13.2 延伸解释 . . . . . | 18        |
| 13.3 练习 . . . . .   | 18        |

# 1 本章导读

本章讨论 **Guest Lecture: Dan Fu 与推理引擎研究**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想训练好的模型要接入真实流量。Serving system (服务系统) 需要调度请求、管理 KV cache、写高效 kernels、处理 prefill/decode，并在 latency、throughput 和成本之间做工程取舍。本章对应 CS336 Spring 2026 第 18 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

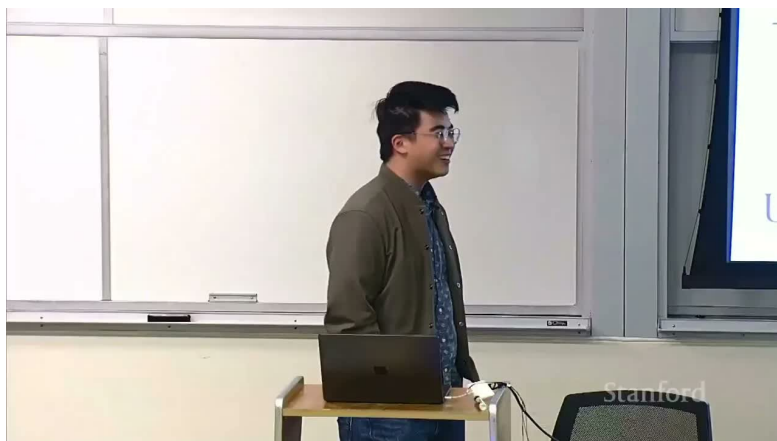


图 1: 第 18 讲的课程图像锚点。

## 1.1 本章知识路线

- 本章首先说明 Guest Lecture: Dan Fu 与推理引擎研究为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

## 1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

# 2 本章主线

本章可以沿着三条线索阅读：课程为什么把这个主题放在这里，它改变了哪些变量，以及怎样用小实验检查这些变量。

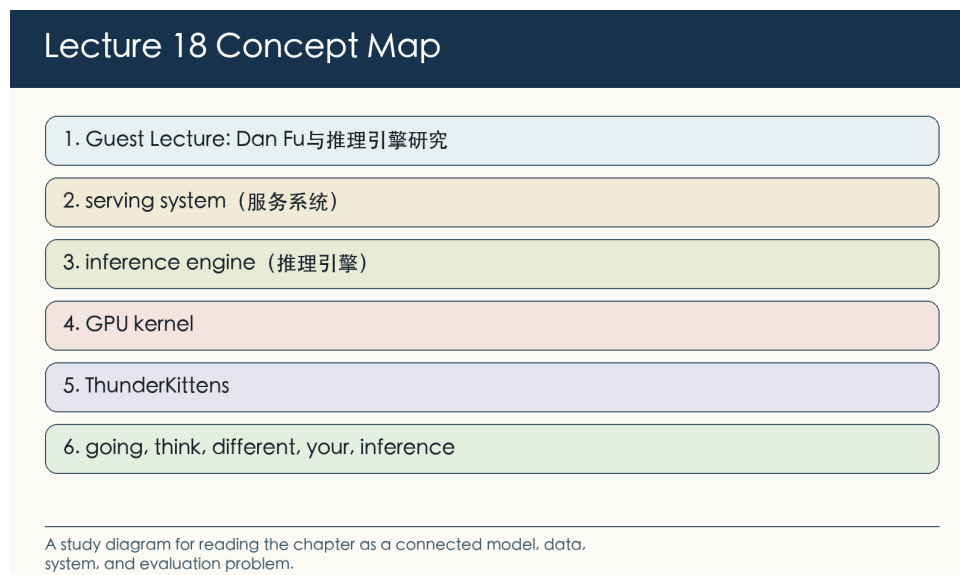


图 2: 第 18 讲概念图: 本章问题、术语和机制的关系。

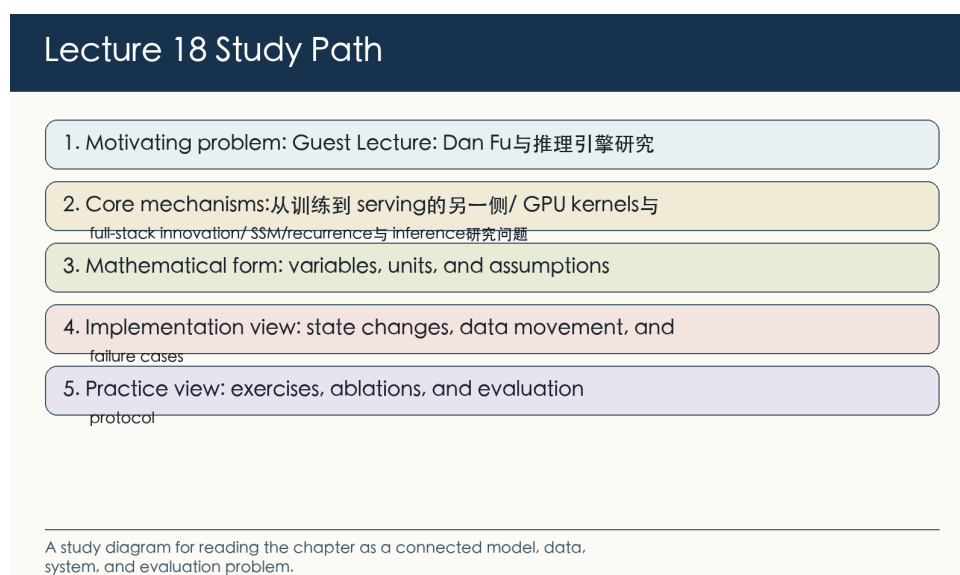


图 3: 第 18 讲学习路径: 从问题定义进入公式、实现、实验和练习。

## 2.1 本章要解决的核心问题

Dan Fu guest lecture 从 serving 侧看语言模型: 训练好的模型要变成真实服务, 需要 inference engine、GPU kernels、scheduler、KV cache 管理和生产流量处理。这一点对应的学习动作是: 找出对象, 写出变量, 说明变量如何被测量。若输入规模、硬件、数据分布或评估协议改变, 结论也必须重新检查。因此, Guest Lecture: Dan Fu 与推理引擎研究不应被读成若干名词的集合, 而应被读成一组可检验的机制。

## 2.2 从机制到资源账本

推理系统是 full-stack innovation 的位置。算法结构、kernel library、memory bandwidth、batching 和 request distribution 会共同决定用户体验。这一点对应的学习动作是: 找出对象, 写出变量, 说明变量如何被测量。若输入规模、硬件、数据分布或评估协议改变, 结论也必须重新检查。因此, Guest Lecture: Dan Fu 与推理引擎研究不应被读成若干名词的集合, 而应被读成一组可检验的机制。

2.3 从课堂讲解到可复现实验

ThunderKittens、megakernel、SSM/recurrence 等讨论说明 serving 不是模型训练后的附属环节，而是会反过来影响模型架构研究的问题。这一点对应的学习动作是：找出对象，写出变量，说明变量如何被测量。若输入规模、硬件、数据分布或评估协议改变，结论也必须重新检查。因此，Guest Lecture: Dan Fu 与推理引擎研究不应被读成若干名词的集合，而应被读成一组可检验的机制。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

| # | 术语                      | 阅读要求                                          |
|---|-------------------------|-----------------------------------------------|
| 1 | serving system（服务系统）    | 给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。           |
| 2 | inference engine（推理引擎）  | 给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。           |
| 3 | GPU kernel              | 把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。 |
| 4 | ThunderKittens          | 给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。           |
| 5 | decode loop（解码循环）       | 给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。           |
| 6 | state-space model (SSM) | 给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。           |

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 关键词

下面的关键词在课程材料中反复出现。它们不代替定义，只提示读者本章哪些对象需要在后文持续跟踪。

- executable, linked, public, playlist, item, spring, page.

4 从训练到 serving 的另一侧

现在进入从训练到 serving 的另一侧。它把 Guest Lecture: Dan Fu 与推理引擎研究中的一部分问题推进到可操作层面：哪些对象要被定义，哪些变量要被测量，哪些限制会改变结论。

4.1 为什么要讨论从训练到 serving 的另一侧

Dan Fu guest lecture 的视频开场说明：课程大多讲如何训练模型，本讲从拥有模型后的另一侧看 inference 和 serving。推理引擎把抽象的模型计算图映射到具体 GPU kernels、调度器、cache 管理和生产流量。字幕中

反复出现 inference、GPU、kernel、decode、cache、request 等词，显示核心主题是 serving stack。从训练到 serving 的另一侧的作用是把 Guest Lecture: Dan Fu 与推理引擎研究中的一个抽象问题变成可计算对象：哪些变量进入公式，哪些状态进入代码，哪些条件会让结果失效。

## 4.2 从训练到 serving 的另一侧的机制

Dan Fu guest lecture 的视频开场说明：课程大多讲如何训练模型，本讲从拥有模型后的另一侧看 inference 和 serving。这句话把讨论落在硬件和系统状态上。读者需要知道哪些量可直接观察，哪些量只能通过日志、profile 或评估协议间接推断。推理引擎把抽象的模型计算图映射到具体 GPU kernels、调度器、cache 管理和生产流量。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。字幕中反复出现 inference、GPU、kernel、decode、cache、request 等词，显示核心主题是 serving stack。这使本节具有可检验性：一个结论若不能在公式、伪代码、profile trace、benchmark 或 ablation 中表现出来，就仍然只是直觉。因此，从训练到 serving 的另一侧应当被理解为可以实现和测试的机制，而不是一个章节标题。CS336 反复强调 from scratch，正是因为只有能被复现的机制，才可能被稳定改进。

## 4.3 从训练到 serving 的另一侧的数学表达

$$\text{user latency} = t_{\text{queue}} + t_{\text{prefill}} + t_{\text{decode}} + t_{\text{postprocess}}$$

符号说明：用户感知延迟由排队、预填充、逐 token 解码和后处理组成；每一项都可能成为瓶颈。这个关系式应被读成从训练到 serving 的另一侧的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。例如，validation loss 常被用作模型质量的 proxy，tokens/s 用作吞吐 proxy，Jaccard 或 MinHash 用作近重复 proxy，reward model score 用作偏好 proxy。proxy 不是目标本身；它只在评估协议清楚时才可用。

## 4.4 从训练到 serving 的另一侧在系统中的实现

把从训练到 serving 的另一侧写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
request = receive_prompt()
kv = prefill(request.prompt)
for step in decode_loop:
    token, kv = engine.decode(kv)
    stream_to_user(token)
```

这段伪代码的读法，是找出从训练到 serving 的另一侧改变了哪些状态，以及这些状态如何对应到吞吐、延迟、显存峰值、通信量和 kernel 利用率。如果某个状态没有被记录，后续实验就很难解释异常结果。

## 4.5 边界条件与常见误解

- 小规模能工作的 serving code 到大规模流量下会暴露罕见 bug。
- 生产 workload 的输入/输出长度分布和教学 benchmark 差异很大。
- 不要把小实验中的局部结论自动外推到 frontier-scale；scale、数据分布和硬件拓扑都会改变结论。
- 不要把 benchmark 或 profile 的单个数字当作机制解释；必须同时记录实验设置、输入形状、版本和评估口径。

## 4.6 与本章其他概念的关系

从训练到 serving 的另一侧与本章其他部分的关系是互补的：术语表给出对象名称，公式给出最小数学结构，伪代码给出执行顺序，例题则把这些内容压缩成一个可检查的计算。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

## 4.7 怎样检验从训练到 serving 的另一侧

检验从训练到 serving 的另一侧时，先把抽象说法落到硬件和系统状态上。与 inference、serving、kernel、decode、request 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：用户感知延迟由排队、预填充、逐 token 解码和后处理组成；每一项都可能成为瓶颈。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。对照实验只应改变一个主要因素，例如 batch size、sequence length、vocabulary size、attention heads、filter threshold、learning rate、decoding 参数或 verifier。多个因素同时变化时，实验最多说明系统表现不同，不能说明是哪一个机制在起作用。最后要主动寻找失败条件。教材中的成功路径容易记住，但工程中真正决定方法边界的是失败案例。本节至少要记住这些限制：小规模能工作的 serving code 到大规模流量下会暴露罕见 bug；生产 workload 的输入/输出长度分布和教学 benchmark 差异很大。复习时可以把这些限制改写成反例测试。实验结论还要放回 Guest Lecture: Dan Fu 与推理引擎研究的整体结构中。从训练到 serving 的另一侧会影响训练目标、数据选择、系统成本或评估解释中的至少一项；能说清楚这种影响，才说明读者已经从名词记忆进入机制理解。

## 4.8 常见混淆：从训练到 serving 的另一侧

本节容易混淆的关键词包括 inference、serving、kernel、decode、request。它们在课程材料中可能连续出现，但层级并不相同：有的命名对象，有的描述操作，有的只是指标或约束。其中，inference（推理）是已训练模型服务请求并生成 token 的过程；serving system（服务系统）把模型计算图变成可承载真实流量的推理引擎。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。如果一个词不能对应到变量、状态、代码路径或评估协议，它在本节中就还没有形成可操作含义。因此，复习从训练到 serving 的另一侧时不应只抄关键词，而应写出至少一个最小例子：输入是什么，哪一步使用这个词，输出或指标怎样变化，失败条件在哪里出现。

## 4.9 从训练到 serving 的另一侧的小结

- 讨论从训练到 serving 的另一侧时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果一个方法声称降低主要成本，要追问成本是否转移到了显存、通信、数据质量、训练稳定性或评估复杂度。
- 公式和伪代码提供推理骨架；结论仍需要 profiler、ablation 或 held-out evaluation 支持。

# 5 GPU kernels 与 full-stack innovation

现在进入 GPU kernels 与 full-stack innovation。它把 Guest Lecture: Dan Fu 与推理引擎研究中的一部分问题推进到可操作层面：哪些对象要被定义，哪些变量要被测量，哪些限制会改变结论。



## 5.1 为什么要讨论 GPU kernels 与 full-stack innovation

讲者强调理解 inference engines 和 GPU kernels 可以开启 full-stack innovation。ThunderKittens 被作为更低层 kernel-writing library 的例子，用于获得更细粒度控制和高带宽利用。Megakernel、fusion、cache-aware scheduling 等方向说明算法和系统可以共同设计。系统部分的核心问题是把数学计算映射到硬件。一个算法的复杂度公式只能给出方向；真正的速度还取决于内存层级、通信拓扑、kernel shape 和调度方式。

## 5.2 GPU kernels 与 full-stack innovation 的机制

讲者强调理解 inference engines 和 GPU kernels 可以开启 full-stack innovation。这句话把讨论落在硬件和系统状态上。读者需要知道哪些量可直接观察，哪些量只能通过日志、profile 或评估协议间接推断。ThunderKittens 被作为更低层 kernel-writing library 的例子，用于获得更细粒度控制和高带宽利用。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。Megakernel、fusion、cache-aware scheduling 等方向说明算法和系统可以共同设计。这使本节具有可检验性：一个结论若不能在公式、伪代码、profile trace、benchmark 或 ablation 中表现出来，就仍然只是直觉。因此，GPU kernels 与 full-stack innovation 应当被理解为可以实现和测试的机制，而不是一个章节标题。CS336 反复强调 from scratch，正是因为只有能被复现的机制，才可能被稳定改进。

## 5.3 GPU kernels 与 full-stack innovation 的资源账本

$$\text{bandwidth utilization} = \frac{\text{bytes served per second}}{\text{peak memory bandwidth}}$$

符号说明：decode 阶段常 memory-bandwidth bound，因此带宽利用率直接影响 tokens/s。这个关系式应被读成 GPU kernels 与 full-stack innovation 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。例如，validation loss 常被用作模型质量的 proxy，tokens/s 用作吞吐 proxy，Jaccard 或 MinHash 用作近重复 proxy，reward model score 用作偏好 proxy。proxy 不是目标本身；它只在评估协议清楚时才可用。

## 5.4 GPU kernels 与 full-stack innovation 在系统中的实现

把 GPU kernels 与 full-stack innovation 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
load_model_tiles()
load_kv_cache_tiles()
run_fused_decode_kernel()
write_next_token_logits()
```

这段伪代码的读法，是找出 GPU kernels 与 full-stack innovation 改变了哪些状态，以及这些状态如何对应到吞吐、延迟、显存峰值、通信量和 kernel 利用率。如果某个状态没有被记录，后续实验就很难解释异常结果。

## 5.5 边界条件与常见误解

- 更低层抽象换来性能，也带来更高 correctness/debug 成本。
- kernel 微小错误在大规模 serving 中可能以极低概率触发，但影响严重。
- 不要把小实验中的局部结论自动外推到 frontier-scale；scale、数据分布和硬件拓扑都会改变结论。

- 不要把 benchmark 或 profile 的单个数字当作机制解释；必须同时记录实验设置、输入形状、版本和评估口径。

## 5.6 与本章其他概念的关系

GPU kernels 与 full-stack innovation 与本章其他部分的关系是互补的：术语表给出对象名称，公式给出最小数学结构，伪代码给出执行顺序，例题则把这些内容压缩成一个可检查的计算。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

## 5.7 怎样检验 GPU kernels 与 full-stack innovation

检验 GPU kernels 与 full-stack innovation 时，先把抽象说法落到硬件和系统状态上。与 ThunderKittens、megakernel、bandwidth、fusion、GPU 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：decode 阶段常 memory-bandwidth bound，因此带宽利用率直接影响 tokens/s。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。对照实验只应改变一个主要因素，例如 batch size、sequence length、vocabulary size、attention heads、filter threshold、learning rate、decoding 参数或 verifier。多个因素同时变化时，实验最多说明系统表现不同，不能说明是哪一个机制在起作用。最后要主动寻找失败条件。教材中的成功路径容易记住，但工程中真正决定方法边界的是失败案例。本节至少要记住这些限制：更低层抽象换来性能，也带来更高 correctness/debug 成本；kernel 微小错误在大规模 serving 中可能以极低概率触发，但影响严重。复习时可以把这些限制改写成反例测试。实验结论还要放回 Guest Lecture: Dan Fu 与推理引擎研究的整体结构中。GPU kernels 与 full-stack innovation 会影响训练目标、数据选择、系统成本或评估解释中的至少一项；能说清楚这种影响，才说明读者已经从名词记忆进入机制理解。

## 5.8 常见混淆：GPU kernels 与 full-stack innovation

本节容易混淆的关键词包括 ThunderKittens、megakernel、bandwidth、fusion、GPU。它们在课程材料中可能连续出现，但层级并不相同：有的命名对象，有的描述操作，有的只是指标或约束。其中，kernel（内核）是高层 tensor 操作落到 GPU 的执行单元；GPU 是 CS336 systems 部分的主要硬件对象。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。如果一个词不能对应到变量、状态、代码路径或评估协议，它在本节中就还没有形成可操作含义。因此，复习 GPU kernels 与 full-stack innovation 时不应只抄关键词，而应写出至少一个最小例子：输入是什么，哪一步使用这个词，输出或指标怎样变化，失败条件在哪里出现。

## 5.9 GPU kernels 与 full-stack innovation 的小结

- 讨论 GPU kernels 与 full-stack innovation 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果一个方法声称降低主要成本，要追问成本是否转移到了显存、通信、数据质量、训练稳定性或评估复杂度。
- 公式和伪代码提供推理骨架；结论仍需要 profiler、ablation 或 held-out evaluation 支持。

## 6 SSM/recurrence 与 inference 研究问题

现在进入 SSM/recurrence 与 inference 研究问题。它把 Guest Lecture: Dan Fu 与推理引擎研究中的一部分问题推进到可操作层面：哪些对象要被定义，哪些变量要被测量，哪些限制会改变结论。

### 6.1 为什么要讨论 SSM/recurrence 与 inference 研究问题

字幕中出现 state space、SSM、recurrence 和 scaling laws，说明 guest lecture 还讨论了模型结构如何影响 serving。recurrent/SSM 风格模型有潜力改变长上下文和 decode 的状态管理方式。但新的结构要同时满足训练稳定性、质量、硬件效率和部署复杂度。推理阶段的关键区别在于请求是在线到达的，输出又是逐 token 生成的。训练时可以用大 batch 摊薄成本；服务时还要同时考虑首 token 延迟、吞吐、显存占用和长尾请求。

### 6.2 SSM/recurrence 与 inference 研究问题的机制

字幕中出现 state space、SSM、recurrence 和 scaling laws，说明 guest lecture 还讨论了模型结构如何影响 serving。这句话把讨论落在模型规模、token 数和训练预算上。读者需要知道哪些量可直接观察，哪些量只能通过日志、profile 或评估协议间接推断。recurrent/SSM 风格模型有潜力改变长上下文和 decode 的状态管理方式。因而同一个方法在不同规模下可能改变不同的变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时，必须同时给出问题规模和实验条件。但新的结构要同时满足训练稳定性、质量、硬件效率和部署复杂度。这使本节具有可检验性：一个结论若不能在公式、伪代码、profile trace、benchmark 或 ablation 中表现出来，就仍然只是直觉。因此，SSM/recurrence 与 inference 研究问题应当被理解为可以实现和测试的机制，而不是一个章节标题。CS336 反复强调 from scratch，正是因为只有能被复现的机制，才可能被稳定改进。

### 6.3 SSM/recurrence 与 inference 研究问题的资源账本

$$h_t = Ah_{t-1} + Bx_t, \quad y_t = Ch_t$$

符号说明：这是 SSM/递推模型的简化形式； $h_t$  是状态， $x_t$  是输入， $y_t$  是输出。这个关系式给出 SSM/recurrence 与 inference 研究问题的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。例如，validation loss 常被用作模型质量的 proxy，tokens/s 用作吞吐 proxy，Jaccard 或 MinHash 用作近重复 proxy，reward model score 用作偏好 proxy。proxy 不是目标本身；它只在评估协议清楚时才可用。

### 6.4 SSM/recurrence 与 inference 研究问题在系统中的实现

把 SSM/recurrence 与 inference 研究问题写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
state = init_state()
for token in sequence:
    state = recurrent_update(state, token)
    logits = readout(state)
```

这段伪代码的读法，是找出 SSM/recurrence 与 inference 研究问题改变了哪些状态，以及这些状态如何对应到参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。如果某个状态没有被记录，后续实验就很难解释异常结果。

## 6.5 边界条件与常见误解

- 递推结构的训练并行性和长程记忆质量需要实证验证。
- serving 系统收益只有在端到端 workload 中测量才可信。
- 不要把小实验中的局部结论自动外推到 frontier-scale; scale、数据分布和硬件拓扑都会改变结论。
- 不要把 benchmark 或 profile 的单个数字当作机制解释; 必须同时记录实验设置、输入形状、版本和评估口径。

## 6.6 与本章其他概念的关系

SSM/recurrence 与 inference 研究问题与本章其他部分的关系是互补的: 术语表给出对象名称, 公式给出最小数学结构, 伪代码给出执行顺序, 例题则把这些内容压缩成一个可检查的计算。对这一类主题, 最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

## 6.7 怎样检验 SSM/recurrence 与 inference 研究问题

检验 SSM/recurrence 与 inference 研究问题时, 先把抽象说法落到模型规模、token 数和训练预算上。与 SSM、recurrence、state、long context、decode 相关的对象必须能被构造、打印、计数或评分; 否则实验只能得到描述性观察, 不能得到可复现结论。一个合适的小实验应当保留本节机制, 同时让参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真; 关键是读者能手算或打印关键中间量, 并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明: 这是 SSM/递推模型的简化形式;  $(h_t)$  是状态,  $(x_t)$  是输入,  $(y_t)$  是输出。如果数量只能间接观测, 就必须说明使用了什么 proxy; 如果使用 benchmark 或 reward, 也要说明协议和随机性。对照实验只应改变一个主要因素, 例如 batch size、sequence length、vocabulary size、attention heads、filter threshold、learning rate、decoding 参数或 verifier。多个因素同时变化时, 实验最多说明系统表现不同, 不能说明是哪一个机制在起作用。最后要主动寻找失败条件。教材中的成功路径容易记住, 但工程中真正决定方法边界的是失败案例。本节至少要记住这些限制: 递推结构的训练并行性和长程记忆质量需要实证验证; serving 系统收益只有在端到端 workload 中测量才可信。复习时可以把这些限制改写成反例测试。实验结论还要放回 Guest Lecture: Dan Fu 与推理引擎研究的整体结构中。SSM/recurrence 与 inference 研究问题会影响训练目标、数据选择、系统成本或评估解释中的至少一项; 能说清楚这种影响, 才说明读者已经从名词记忆进入机制理解。

## 6.8 常见混淆: SSM/recurrence 与 inference 研究问题

本节容易混淆的关键词包括 SSM、recurrence、state、long context、decode。它们在课程材料中可能连续出现, 但层级并不相同: 有的命名对象, 有的描述操作, 有的只是指标或约束。其中, decode (解码) 逐 token 生成, 常受 memory bandwidth 和 KV cache 影响。区分这些词时, 可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。如果一个词不能对应到变量、状态、代码路径或评估协议, 它在本节中就还没有形成可操作含义。因此, 复习 SSM/recurrence 与 inference 研究问题时不应只抄关键词, 而应写出至少一个最小例子: 输入是什么, 哪一步使用这个词, 输出或指标怎样变化, 失败条件在哪里出现。

## 6.9 SSM/recurrence 与 inference 研究问题的小结

- 讨论 SSM/recurrence 与 inference 研究问题时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果一个方法声称降低主要成本，要追问成本是否转移到了显存、通信、数据质量、训练稳定性或评估复杂度。
- 公式和伪代码提供推理骨架；结论仍需要 profiler、ablation 或 held-out evaluation 支持。

## 7 综合讨论：从局部机制到完整系统

本章的主题是 Guest Lecture: Dan Fu 与推理引擎研究，但它不应被读成几个相邻知识点的拼接。从训练到 serving 的另一侧、GPU kernels 与 full-stack innovation、SSM/recurrence 与 inference 研究问题共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 serving system（服务系统）、inference engine（推理引擎）、GPU kernel、ThunderKittens、decode loop（解码循环）、state-space model (SSM)。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

### 7.1 从训练到 serving 的另一侧在系统中的位置

从训练到 serving 的另一侧首先提供一个局部解释：Dan Fu guest lecture 的视频开场说明：课程大多讲如何训练模型，本讲从拥有模型后的另一侧看 inference 和 serving。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从训练到 serving 的另一侧会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从训练到 serving 的另一侧不能脱离边界条件理解。一个关键 caveat 是：小规模能工作的 serving code 到大规模流量下会暴露罕见 bug。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

### 7.2 GPU kernels 与 full-stack innovation 在系统中的位置

GPU kernels 与 full-stack innovation 首先提供一个局部解释：讲者强调理解 inference engines 和 GPU kernels 可以开启 full-stack innovation。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，GPU kernels 与 full-stack innovation 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，GPU kernels 与 full-stack innovation 不能脱离边界条件理解。一个关键 caveat 是：更低层抽象换来性能，也带来更高 correctness/debug 成本。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

### 7.3 SSM/recurrence 与 inference 研究问题在系统中的位置

SSM/recurrence 与 inference 研究问题首先提供一个局部解释：字幕中出现 state space、SSM、recurrence 和 scaling laws，说明 guest lecture 还讨论了模型结构如何影响 serving。这类解释的价值在于把直觉压缩成

可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，SSM/recurrence 与 inference 研究问题会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，SSM/recurrence 与 inference 研究问题不能脱离边界条件理解。一个关键 caveat 是：递推结构的训练并行性和长程记忆质量需要实证验证。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

## 7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

## 8 例题：把概念变成可计算检查

### 8.1 Serving latency 的分解

一个用户请求从进入队列到 token 流式返回。

- 排队时间取决于调度和 batching。
- prefill 处理 prompt，decode 循环逐 token 运行。
- KV cache 管理、kernel 效率和输出长度共同影响 tail latency。

**结论。** serving 研究把模型结构、GPU kernels 和系统调度连成一条端到端路径。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

## 9 实验设计：从小例子到可复现结论

学习 Guest Lecture: Dan Fu 与推理引擎研究不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

## 9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

## 9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

## 9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

## 9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

## 9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

## 9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

## 10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

| 时间                | 关键词                                                        | 复习任务                        |
|-------------------|------------------------------------------------------------|-----------------------------|
| 00:00:05-00:06:23 | think, inference, horses, pretty, understand, code, around | 回看定义、例子、推导或 caveat 在该段中的位置。 |
| 00:17:45-00:23:43 | different, very, then, compute, prefill, decode, start     | 回看定义、例子、推导或 caveat 在该段中的位置。 |
| 00:35:36-00:41:54 | your, attention, kernel, start, very, then, operation      | 回看定义、例子、推导或 caveat 在该段中的位置。 |
| 00:47:41-00:53:39 | going, matrix, there's, look, then, it's, said             | 回看定义、例子、推导或 caveat 在该段中的位置。 |
| 01:05:14-01:11:14 | think, then, your, size, cache, going, choices             | 回看定义、例子、推导或 caveat 在该段中的位置。 |

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

| 资料组         | 标题/页块                                                                                     | 关键词                                                           |
|-------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| official_01 | No official slide PDF or executable lecture script is linked for this public playlist ... | official, slide, executable, script, linked, public, playlist |

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Guest Lecture: Dan Fu 与推理引擎研究在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。



- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution; 脱离 workload 的性能数字不可直接复用。

## 12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

| 条目   | 对象                      | 检查动作                                              |
|------|-------------------------|---------------------------------------------------|
| 条目 1 | serving system (服务系统)   | 定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。 |
| 条目 2 | inference engine (推理引擎) | 定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。 |
| 条目 3 | GPU kernel              | 定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。 |
| 条目 4 | ThunderKittens          | 定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。 |
| 条目 5 | decode loop (解码循环)      | 定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。 |
| 条目 6 | state-space model (SSM) | 定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。 |

### 12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

## 13 总结与延伸

### 13.1 本章小结

- Guest Lecture: Dan Fu 与推理引擎研究的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

## 13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

## 13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。